PlayEarning Nexus — Master Launch Guide (Updated)

Everything to configure, build, and ship — in the order to do it

Updated July 24, 2026. This is the single, end-to-end guide to take the app from code to live on web, Android, and iOS. It's ordered by best practice and efficiency: set up accounts → wire APIs → build → PWA → deploy → automate → legal → native apps → go live.

This app is fully self-hosted — it no longer uses Base44. The backend runs on your own Deno + PostgreSQL stack.

Repository status (verified 2026-07-24): ✅ All code is on GitHub `main` and current — the self-hosted backend, the frontend, the Capacitor mobile wrapper, the in-app legal pages, and all docs. Repo: `https://github.com/benjaminjohnvick-cmyk/playearningnexus`.

Code audit (2026-07-24): A full static audit (TypeScript compiler + ESLint over ~230k lines + a security-pattern battery) was run and all findings fixed and pushed — including a SQL-injection vector in the data layer, an admin auth-bypass, dead Base44 URLs in two emails, and the AI-order-fulfillment margin/email bugs. Details in `FULL-CODE-AUDIT-0724.md`. Still open (not blocking): server-side card-payment verification in `placeStoreOrder`, and sanitizing user-derived HTML in the three `dangerouslySetInnerHTML` spots.

Companion docs: `CONFIG-AND-SECRETS.md`, `SETUP-RUNBOOK.md`, `backend/PHASE-2-RUNBOOK.md`, `DEVELOPER-HANDOFF-BRIEF.md`, `MOBILE-APP-WRAPPER-GUIDE.md`, `APP-STORE-SUBMISSION-CHECKLIST.md`, `LEGAL-PAGES-GUIDE.md`, `PRIVACY-POLICY.md`, `TERMS-OF-SERVICE.md`, `COMPLIANCE-AND-ASSUMPTIONS.md`, `DE-BASE44-REWORK.md`.

How the app is built (read first)

- **Backend** = a **self-hosted Deno service** (`/backend`). It runs the 526 functions (as HTTP routes), 239 PostgreSQL tables, JWT + Google authentication, an agent runtime, a cron scheduler, and the AI/email/image integrations. You deploy it as a container; it scales with your host. **Secrets live in your backend host's environment**, never in the repo.
- **Database** = **PostgreSQL** (schema in `backend/db/schema.sql`). Use a managed provider (Neon, Supabase, RDS) or your own.
- **Frontend** = a **React + Vite PWA** (208 pages). Public `VITE_*` values are build-time. It deploys as a static site and wraps to native via **Capacitor**. It points at the backend via `VITE_NEXUS_API_URL`.
- **Mobile** = **wrapper-only**. There are **no `android/` / `ios/` folders in the repo** — they're git-ignored and regenerated on demand with `npm run native:regenerate`. All native behavior lives in the web layer (`src/lib/native.js`).
- **Rule:** if it's a *secret*, it goes in the **backend host's env** (`backend/.env`). If it starts with `VITE_`, it's public and goes in the **frontend build** (`.env.local`).

> **New with self-hosting:** the AI/email/image features (`InvokeLLM`, `SendEmail`, `GenerateImage`, etc.) used to run free on Base44's credentials — now they use **your own** OpenAI/Anthropic, SendGrid/SES, and S3 keys. LLM usage is the main variable cost; budget for it.

LAUNCH SEQUENCE AT A GLANCE

1. Create accounts (Phase 1)
2. Get all API keys (Phase 2 — the complete list)
3. Configure keys — backend host + frontend (Phase 3)
4. Build & smoke-test (Phase 4)
5. Finish the PWA — icons + install test (Phase 5)
6. Deploy the web app — host, domain, HTTPS (Phase 6)

7. Turn on backend automation schedules (Phase 7)
8. Legal & compliance (Phase 8)
9. Build the native apps (Phase 9)
10. Final QA & go live (Phase 10)

PHASE 1 — Accounts to create first

Account	For	Cost
Backend host (Render / Railway / Fly.io / AWS)	Runs the Deno backend container	pay-as-you-go
Managed Postgres (Neon / Supabase / RDS)	The database	free tier → pay-as-you-go
OpenAI or Anthropic	LLM features (was free via Base44)	usage-based
SendGrid or AWS SES	Email (reset/invite/notifications)	free tier → usage
AWS S3 (or compatible)	File uploads	pay-as-you-go
Google Cloud (optional)	"Sign in with Google" OAuth client	free
Stripe	Card payments	free; fees per txn
PayPal Developer (Business)	PayPal payments + payouts	free; fees
BitLabs	Third-party survey supply	revenue share
Twilio	SMS notifications	pay-as-you-go
Meta for Developers	Facebook + Instagram login/posting	free
X (Twitter) Developer	Twitter/X posting	free tier/paid
Snap Kit (Snapchat)	Snapchat integration	free
ScrapingBee + **Browserless**	Competitive-intel scraping	paid tiers (optional at launch)
AWS (or Amplify)	Frontend hosting	pay-as-you-go
Domain registrar	Your custom domain	~$12/yr
Google Play Console	Android app	$25 one-time
Apple Developer Program	iOS app (needs a Mac)	$99/yr

PHASE 2 — COMPLETE API / KEY LIST

Everything the code references. "Where" = backend host env (`backend/.env`), or frontend `.env`. "Priority" = needed to launch vs. can follow.

A. Backend secrets — set in your backend host's env / `backend/.env` (never in the repo)

Self-hosted platform (new — these replace what Base44 used to provide):
Key	Powers	Where to get it	Priority
`DATABASE_URL`	PostgreSQL connection	Your managed Postgres provider	**Required**
`AUTH_JWT_SECRET`	Signs user login tokens	Generate a long random string	**Required**
`OPENAI_API_KEY` (or `ANTHROPIC_API_KEY`)	LLM / agents / image / speech	OpenAI or Anthropic dashboard	**Required** (AI features)
`SENDGRID_API_KEY` (or SES/SMTP)	Email (reset, invites, notifications)	SendGrid, or AWS SES	**Required** (auth email)
`AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` / `S3_BUCKET` / `AWS_REGION`	File uploads → S3	AWS IAM + S3	High
`GOOGLE_CLIENT_ID`	Verify "Sign in with Google"	Google Cloud Console	Optional

| `FRONTEND_URL` | Password-reset links | Your frontend domain | **Required** |

Your app integrations:
Key	Powers	Where to get it	Priority
`STRIPE_SECRET_KEY`	Card charges/payouts	Stripe → Developers → API keys (Secret `sk_live_…`)	**Required**
`PAYPAL_CLIENT_ID`	PayPal API	PayPal Developer → Apps & Credentials	**Required**
`PAYPAL_SECRET_KEY`	PayPal API	PayPal Developer → Apps → Secret	**Required**
`PAYOUT_WEBHOOK_URL`	Payout webhook target	Your endpoint URL	**Required** if using payout webhooks
`PAYOUT_WEBHOOK_SECRET`	Verify payout webhooks	A random secret you generate	**Required** if using payout webhooks
`BITLABS_API_KEY`	External survey offers	BitLabs dashboard → API	**Required** (core earning)
`TWILIO_ACCOUNT_SID`	SMS	Twilio Console	High (SMS features)
`TWILIO_AUTH_TOKEN`	SMS	Twilio Console	High
`TWILIO_PHONE_NUMBER`	SMS sender	Twilio → Phone Numbers	High
`VAPID_PUBLIC_KEY`	Web push	Generate: `npx web-push generate-vapid-keys`	High
`VAPID_PRIVATE_KEY`	Web push	Same command (keep private)	High
`FACEBOOK_APP_ID` / `FACEBOOK_APP_SECRET`	FB login + posting	Meta for Developers → your app	Medium
`INSTAGRAM_APP_ID` / `INSTAGRAM_APP_SECRET`	IG posting	Meta for Developers → your app	Medium
`TWITTER_API_KEY` / `TWITTER_API_SECRET`	X posting	X Developer Portal	Medium
`SNAPCHAT_CLIENT_ID` / `SNAPCHAT_CLIENT_SECRET`	Snapchat	Snap Kit portal	Low
`SCRAPINGBEE_API_KEY`	Web scraping (intel)	ScrapingBee	Low (optional)
`BROWSERLESS_API_KEY`	Headless browser	Browserless.io	Low (optional)
`APP_URL`	Absolute links in emails/webhooks	Your production URL	**Required**

B. Frontend variables — set at build time (`.env.local` / host env)

Key	Powers	Where to get it	Priority
`VITE_NEXUS_API_URL`	Your backend URL	Your deployed backend (e.g. `https://api.yourdomain.com`)	**Required**
`VITE_GOOGLE_CLIENT_ID`	"Continue with Google" button	Google Cloud Console (same client id)	Optional
`VITE_STRIPE_PUBLISHABLE_KEY`	Stripe checkout (public)	Stripe → API keys (`pk_live_…`)	**Required**
`VITE_PAYPAL_CLIENT_ID`	PayPal buttons (public)	PayPal Developer	**Required**
`VITE_VAPID_PUBLIC_KEY`	Web push (public half)	Same VAPID pair as backend	High

C. AI / email / image / file — now use YOUR keys (no longer free via Base44)

`InvokeLLM`, `GenerateImage`, `GenerateSpeech`, `SendEmail`, `UploadFile` used to run on Base44's platform credentials. Self-hosted, they use your own: **LLM** → `OPENAI_API_KEY` / `ANTHROPIC_API_KEY`; **email** → SendGrid/SES; **uploads** → S3 (all in Phase 2-A above). LLM usage is the main variable cost.

D. Free / no-key external services (already working)

- **Maps:** React Leaflet + OpenStreetMap tiles — no key.
- **Currency rates:** exchangerate-api.com free endpoint — no key.

E. Payout methods present in code

PayPal and Stripe (keyed above). **Venmo** and **Cash App** payout functions exist; these typically route through PayPal/manual review rather than a separate API key — confirm your payout operations before enabling.

> Full details and exact variable names: `CONFIG-AND-SECRETS.md` and `SETUP-RUNBOOK.md`.

PHASE 3 — Configure the keys

1. **Backend:** set every key from Phase 2-A in your backend host's environment (or `backend/.env` locally). Start from `backend/.env.example`.
2. **Frontend:** create `.env.local` in the repo root with the Phase 2-B values (and set the same in your host's build settings later). Example:

```
VITE_NEXUS_API_URL=https://api.yourdomain.com
VITE_GOOGLE_CLIENT_ID=xxx            # optional (Google sign-in)
VITE_STRIPE_PUBLISHABLE_KEY=pk_live_xxx
VITE_PAYPAL_CLIENT_ID=xxx
VITE_VAPID_PUBLIC_KEY=xxx
```

3. **Deploy the backend** container and load `backend/db/schema.sql` into your Postgres so functions/tables/agents are live. (Locally: `cd backend && docker compose up`.)

PHASE 4 — Build & smoke-test

```
git clone https://github.com/benjaminjohnvick-cmyk/playearningnexus.git
cd playearningnexus
npm install
npm run build        # must produce ./dist with no errors
npm run lint         # optional but recommended
npm run preview      # open the built app locally and click through
```

Verify: sign-in works, a Stripe/PayPal **test-mode** transaction completes, a survey/referral action credits.

PHASE 5 — Finish the PWA (works on Android + iOS)

The manifest, mobile meta tags, and a **placeholder icon set** are already in the repo. Remaining:
1. **Swap in your real icon.** Replace `assets/icon.png` with your final **1024×1024** PNG, then run `npm run cap:assets` to regenerate `public/icons/icon-192.png`, `icon-512.png`, `icon-512-maskable.png`, and `apple-touch-icon.png`. (Placeholders ship now so it builds; use your real logo before launch.)
2. **Confirm the service worker** is registered and caching what you want.
3. **Test install:** deploy (Phase 6), then on **Android Chrome** use "Add to Home screen / Install app"; on **iOS Safari** use Share → "Add to Home Screen." Confirm it opens full-screen with your icon and name.

> A well-formed PWA installs on both platforms from the browser. For **app-store** distribution, do Phase 9.

PHASE 6 — Deploy the web app

1. Build output is `./dist` (static).
2. Host on **AWS Amplify Hosting** (recommended: CDN + auto-scaling + CI/CD) or **S3 + CloudFront**.
3. **SPA history fallback (required):** the app uses `BrowserRouter`, so redirect `404/403 → /index.html` (Amplify: a single rewrite rule; S3/CloudFront: error-document or a CloudFront function). Without this, deep links break.
4. Set the `VITE_*` env vars in the host's build settings.
5. **Custom domain + HTTPS:** attach your domain and an ACM certificate.
6. Point `VITE_NEXUS_API_URL` (frontend) and `APP_URL` / `FRONTEND_URL` (backend) at production.
7. **Confirm the public legal URLs resolve** (needed for the app stores): `https://yourdomain.com/PrivacyPolicy` and `/TermsOfService`.

PHASE 7 — Turn on backend automation (schedules)

Your automation functions run on a schedule via the included scheduler (`backend/scheduler`, uses `Deno.cron`) or your host's cron / AWS EventBridge. Recommended cadence:

- **Daily:** `autoReferralContestDaily`, `autonomousEcosystemEngine` (if `EcosystemConfig.autonomous_mode` = true), `creditPendingReferralPostRewards` (grace-period sweep), `autoDailyOperationsEngine`, `generateAIDailyGoal`, survey generation.
- **Weekly:** `processWeeklyJackpot` (the merit-based, open, self-funding prize pool), `generateWeeklyReferralCampaign` + `concludeWeeklyReferralCampaign`, `generateWeeklyFeatureVoteSurvey` + `concludeWeeklyFeatureVote`, `weeklyContestWinner`, `autoWeeklyReportsEngine`.
- **Every 6–12h:** `masterOrchestrator`, `aiOrchestrator`.
- **Set platform config:** `GlobalSettings` (e.g., prize-pool contribution), admin credentials, and default `EcosystemConfig` / `ReferralJackpot` values (entry fee, `pool_funding_rate`).

PHASE 8 — Legal & compliance (do before public launch)

The in-app pages exist (`src/pages/PrivacyPolicy.jsx`, `src/pages/TermsOfService.jsx`) and matching docs (`PRIVACY-POLICY.md`, `TERMS-OF-SERVICE.md`) — but they are **templates with `[BRACKET]` placeholders and must be completed and lawyer-reviewed**. See `LEGAL-PAGES-GUIDE.md`. From `COMPLIANCE-AND-ASSUMPTIONS.md`:
- **Privacy Policy + Terms of Service** — fill in every placeholder, publish at public URLs, have a lawyer review. Required by app stores and law.
- **Data privacy:** GDPR/CCPA consent capture + opt-out (you collect demographics and survey data).
- **FTC disclosures** on referral/affiliate posts (already enforced in code as `#ad`).
- **Contest rules** for the **merit-based** prize pool + feature votes (official rules, eligibility, state gating). Note: the prize pool is skill/merit-based and open to all — **not** a random draw — which is deliberate for legality; keep it that way.
- **Money-transmitter/escrow review** before enabling real-cash pooling in Shared Wallet Groups (currently closed-loop credits).
- **MLM/referral commission review** (pyramid-scheme rules — tie rewards to real sales).
- **Payments:** confirm Stripe/PayPal accounts verified, tax reporting (1099 thresholds), and payout compliance.

PHASE 9 — Native apps (Android + iOS)

Follow `MOBILE-APP-WRAPPER-GUIDE.md`. The repo is **wrapper-only** (Capacitor configured; native folders regenerated, not committed). Summary:
1. `npm install` → put your **1024×1024** icon at `assets/icon.png`.
2. `npm run native:regenerate` — one command: builds `dist/`, generates icons, creates the `android/` (and, on a Mac, `ios/`) shells, and syncs. Re-run this whenever you build; never commit the generated folders.
3. Open in **Android Studio** (`npm run cap:open:android`) / **Xcode** (`npm run cap:open:ios`, **Mac only**), set version + signing, build a signed `.aab` / archive.
4. Submit to **Google Play** and **App Store** — see the full `APP-STORE-SUBMISSION-CHECKLIST.md` before you upload.

> Store-review watch-outs: Apple Guideline 4.2 (not "just a website"), and extra scrutiny on **earning/rewards/payments** apps. In your Apple review notes, explain that the prize pool and referral rewards are **skill/merit-based, not gambling**, and include a demo login. **Keep your Android signing keystore safe** — you need it for every future update.

PHASE 10 — Final QA & go-live checklist

- [ ] Clean `npm run build` (no errors)
- [ ] All **Required** keys set in the backend host env + frontend
- [ ] Backend **published**; entities/functions/agents live
- [ ] Sign-in / OAuth works in production
- [ ] Real (small) Stripe + PayPal transaction succeeds
- [ ] Payout path tested end-to-end
- [ ] Survey → credit flow works (internal + BitLabs)
- [ ] Referral post + reward credit works (with `#ad` disclosure)
- [ ] Web push (and SMS if used) fire
- [ ] PWA installs on Android + iOS; deep links work (SPA fallback OK)
- [ ] Automation schedules enabled (Phase 7)
- [ ] Privacy Policy + Terms **completed, lawyer-reviewed, live, and linked**

- [ ] Custom domain + HTTPS active
- [ ] Real app icon swapped in (not the placeholder)
- [ ] Error monitoring in place (optional: add Sentry/logging)
- [ ] `APP-STORE-SUBMISSION-CHECKLIST.md` completed (if launching on stores)
- [ ] Native apps approved (if launching on stores)

What's already done (so you don't redo it)

- ✅ Full codebase (208 pages, 526 functions, 235 databases, 76 agents) — built and **verified live on GitHub** `main`.
- ✅ All API integrations **wired in code** (every key above is already read via `Deno.env.get` / `import.meta.env`) — you only supply values.
- ✅ PWA manifest + mobile meta tags + service worker + **placeholder icon set**.
- ✅ **Wrapper-only** Capacitor setup for Android/iOS (`npm run native:regenerate`; native folders git-ignored).
- ✅ Compliance changes (opt-in participation, FTC `#ad` disclosure, **merit-based open self-funding prize pool**, closed-loop wallets).
- ✅ In-app **Privacy Policy** and **Terms** pages (template state — needs your details + lawyer review).
- ✅ Docs: this guide + config, setup, wrapper, legal, compliance, app-store checklist, and push-steps references.

What only you can do (needs your accounts/machine)

- Supply the real API key **values** (Phase 2–3).
- Deploy the backend container + Postgres, and turn on the scheduler (Phase 7).
- Deploy the frontend + domain (Phase 6).
- Complete + legally review the Privacy Policy and Terms (Phase 8).
- Swap in the real app icon and generate/submit the native apps — iOS requires a **Mac** (Phase 9).

Quick reference — the commands you'll actually run

```
# Get the code
git clone https://github.com/benjaminjohnvick-cmyk/playearningnexus.git
cd playearningnexus && npm install

# Web: build, check, preview
npm run build && npm run preview

# Icons (after placing assets/icon.png, 1024x1024)
npm run cap:assets

# Native apps (wrapper-only; regenerates android/ and ios/)
npm run native:regenerate
npm run cap:open:android      # Android Studio
npm run cap:open:ios          # Xcode (Mac only)
```